

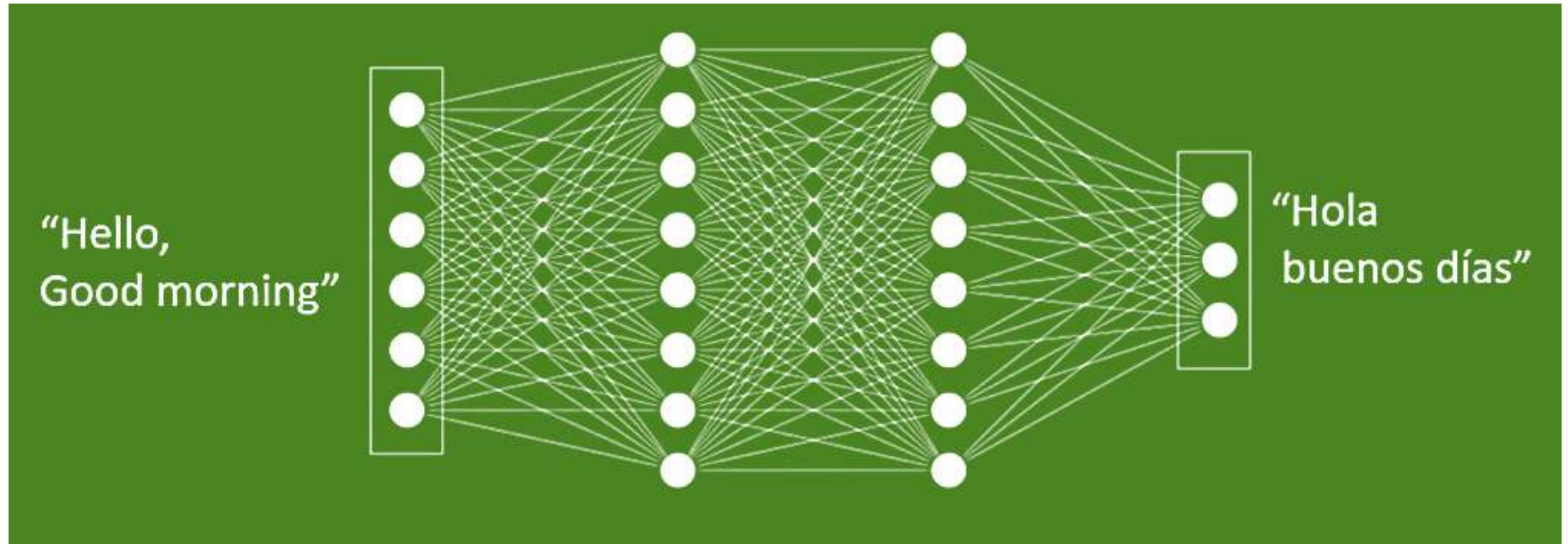
IT3071 – Machine Learning and Optimization Methods

Introduction to Recurrent Neural Network (CNN)

H.M. Samadhi Chathuranga Rathnayake

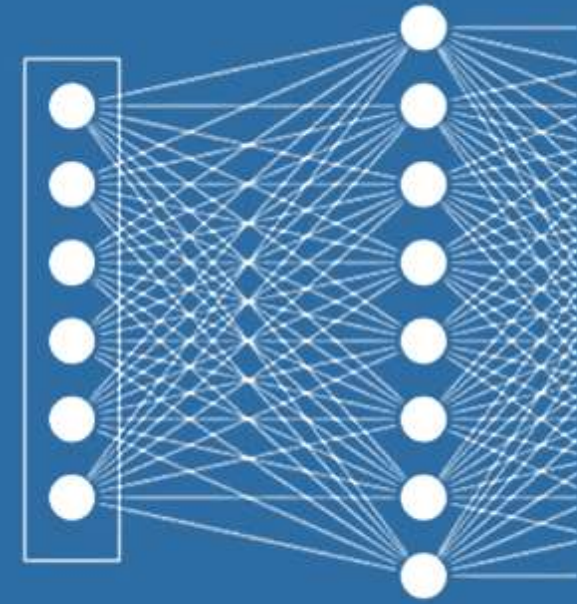
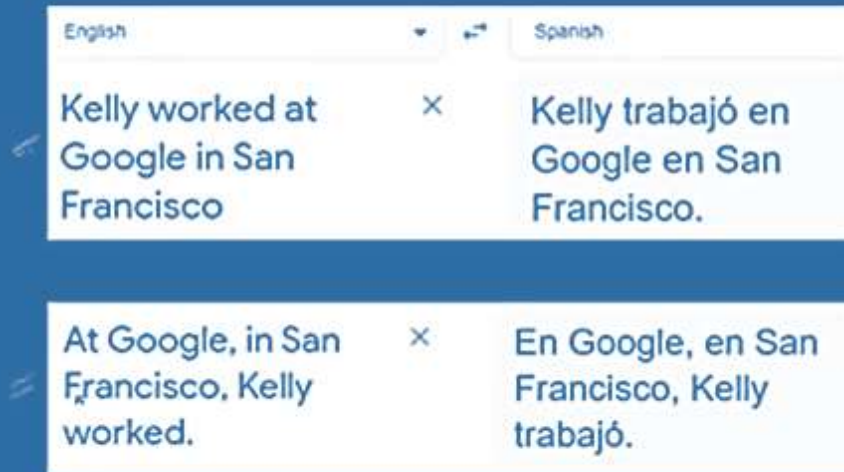
Text Data in Feedforward Neural Networks

- In the feedforward networks, there is a fixed length for the inputs



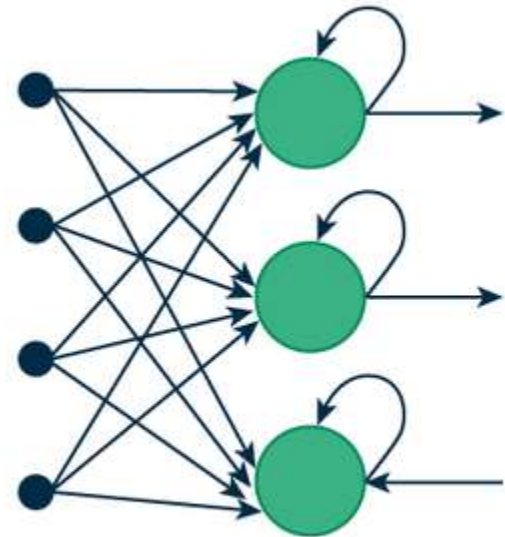
Text Data in Feedforward Neural Networks

- The order is not considered in the feedforward networks

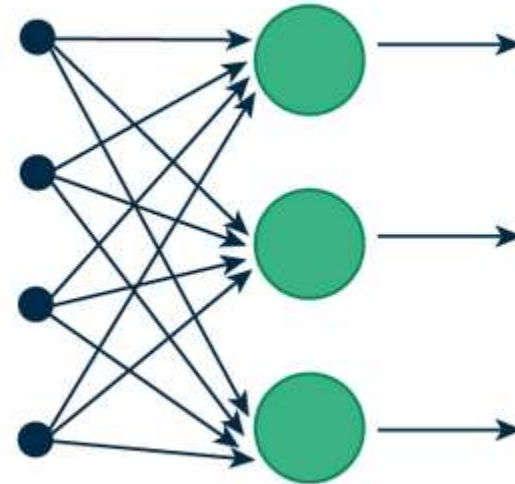


Introduction to Recurrent Neural Networks

- A partial output of the activation functions of the hidden layer will be taken as an input for the next iteration



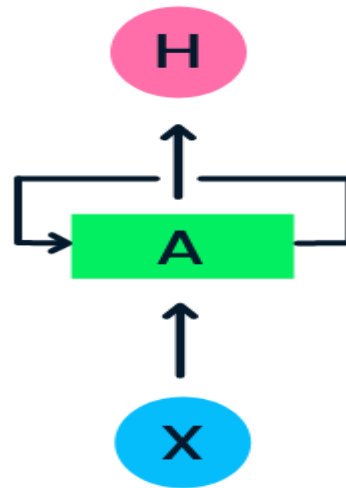
(a) Recurrent Neural Network



(b) Feed-Forward Neural Network

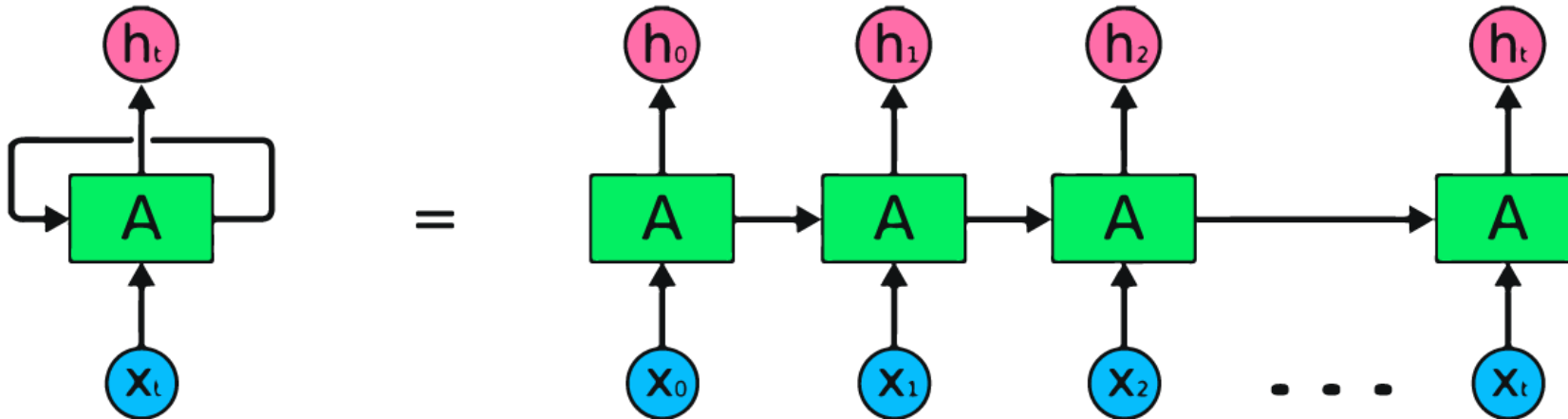
How Recurrent Neural Networks Work?

- In RNN, the information cycles through the loop, so the output is determined by the current input and previously received inputs.



How Recurrent Neural Networks Work?

- In a traditional neural network, you might have multiple hidden layers between the input and output layers, and each hidden layer would have its own separate parameters (like weights and biases).
- However, in a Recurrent Neural Network (RNN), things work a bit differently, especially when processing sequences of data (like time steps in a series or words in a sentence).



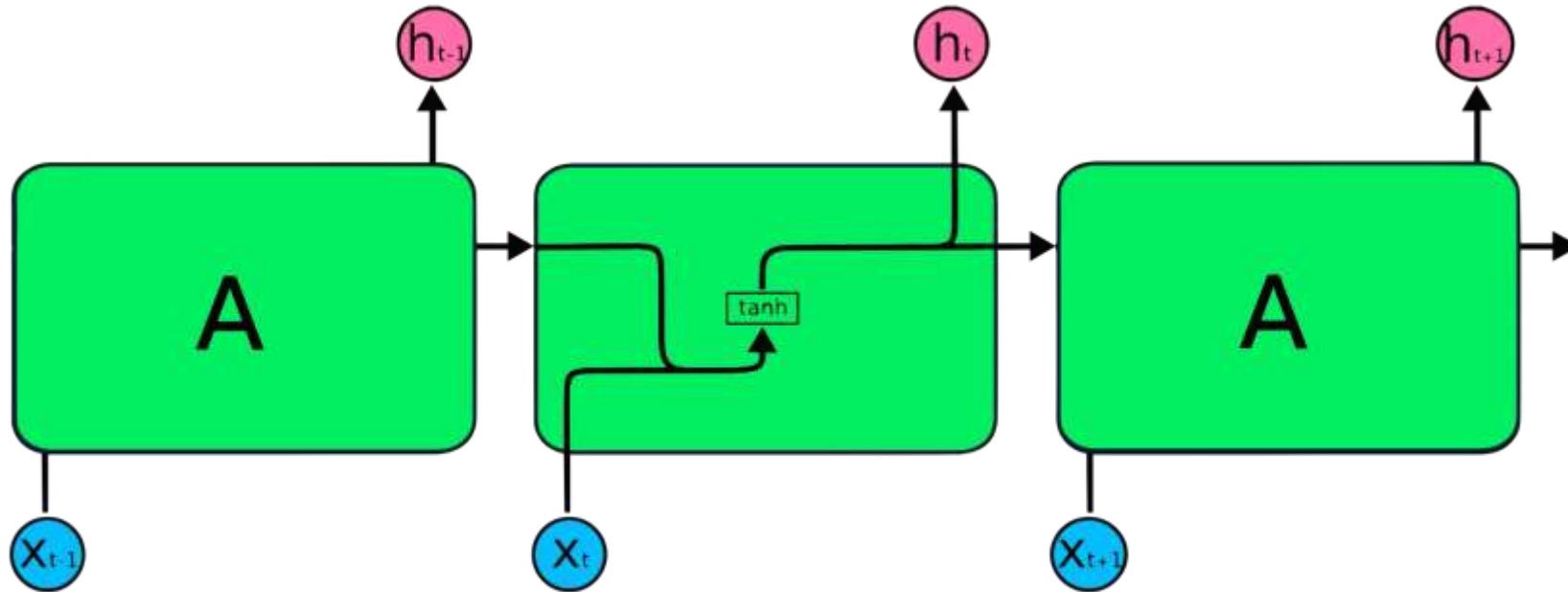
How Recurrent Neural Networks Work?

- Single Input (X):
 - Imagine that the input (X) is not just one piece of data but a sequence.
 - For example, if you're processing a sentence, each word in the sentence is part of that sequence.
 - At each time step in the sequence, the RNN processes one piece of input (like a word or a time step in a time series).
- Middle Layer (A) – Looping for Each Time Step:
 - Instead of creating a new hidden layer for each step in the sequence, the RNN uses the same hidden layer repeatedly.
 - This means that for each time step in the input sequence, the same hidden layer (A) is used.
 - This hidden layer shares the same parameters (weights, biases, and activation function) across all time steps.

How Recurrent Neural Networks Work?

- Ex-
- Suppose your sequence is made up of three words, and you pass the first word to the hidden layer (A).
- The RNN processes it, then moves to the next word and uses the same hidden layer (with the same weights, biases, etc.) to process that word.
- This repetition continues for the entire sequence (for each word).

How Recurrent Neural Networks Work?



How Recurrent Neural Networks Work?

- In an RNN, the hidden state a_t at time t can be expressed as:

$$a_t = f(W_a a_{t-1} + W_x x_t + b_a)$$

- W_a is the weight matrix for the previous hidden state a_{t-1} .
- W_x is the weight matrix for the current input x_t .
- b_h is the bias term.
- f is an activation function, often a non-linear function such as tanh or ReLU.

How Recurrent Neural Networks Work?

- Output in each x_t is,

$$h_t = f'(W_h a_t + b_h)$$

- Common activation functions (f') for different types of output include:
 - Softmax
 - Sigmoid
 - Linear Activation

Why Use the Same Hidden Layer Repeatedly?

- The idea behind using the same hidden layer multiple times is to allow the RNN to remember what happened in previous time steps.
- This is useful because it helps the network maintain a memory of past information while processing the current time step, which is important for sequences where the context matters (like sentences or time series data).

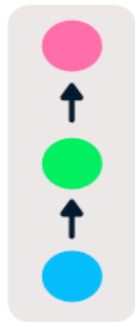
Backpropagation Through Time (BPTT)

- Since the same hidden layer (A) is used for each time step, the model needs a way to adjust the parameters not just for each layer, but also across time steps.
- That's where Backpropagation Through Time (BPTT) comes in.
- BPTT calculates the errors across all time steps in the sequence and adjusts the shared parameters based on the accumulated errors from each step.

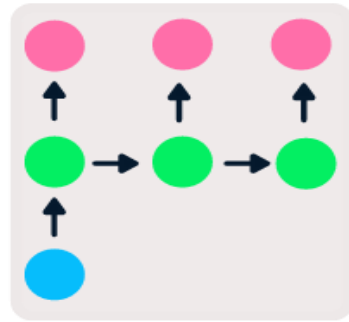
Types of RNN

- Feedforward networks have single input and output, while recurrent neural networks are flexible as the length of inputs and outputs can be changed.
- This flexibility allows RNNs to generate music, sentiment classification, and machine translation.
- There are four types of RNN based on different lengths of inputs and outputs.

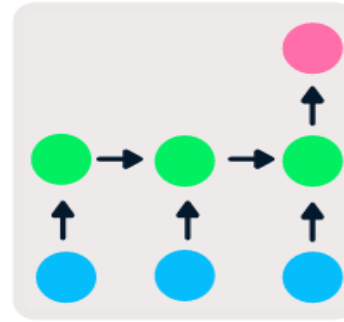
One to One



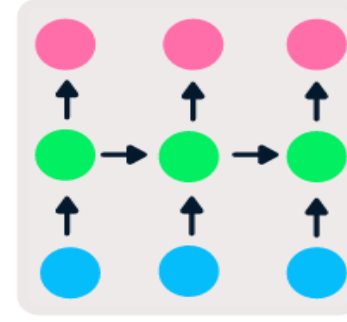
One to Many



Many to One



Many to Many



Types of RNN

- One-to-one (Vanilla RNN) is a simple neural network. It is commonly used for machine learning problems that have a single input and output.
- One-to-many has a single input and multiple outputs. This is used for generating image captions.
- Many-to-one takes a sequence of multiple inputs and predicts a single output. It is popular in sentiment classification, where the input is text and the output is a category.
- Many-to-many takes multiple inputs and outputs. The most common application is machine translation.

Vanishing Gradient Problem

- In RNNs, when the model is training, it adjusts its weights using a process called backpropagation through time.
- In this process, the gradients (or error signals) are calculated and used to update the weights.
- However, when the network is very deep or has long sequences, the gradients can become very small (almost zero) as they are passed backward through the layers.
- This happens when the weights are less than 1.
- When the gradients are too small, the network struggles to update the weights, especially in earlier layers (or earlier time steps).
- This means that the RNN "forgets" important information from the beginning of the sequence.

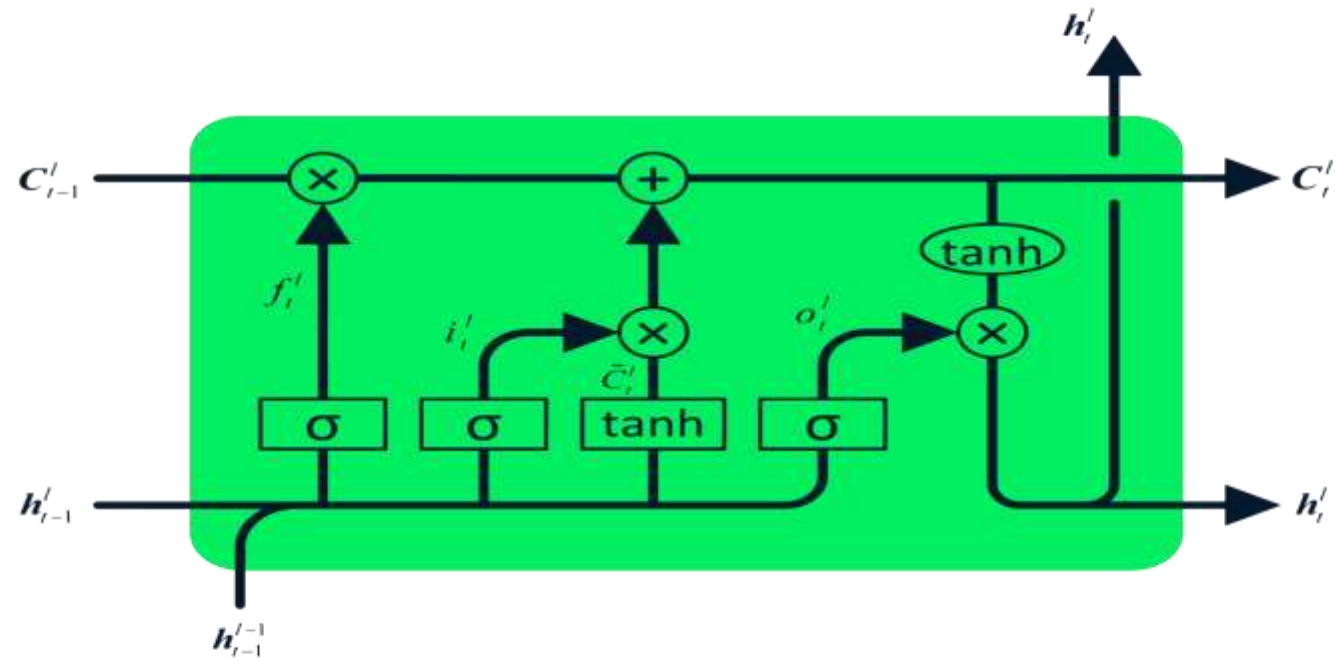
Vanishing Gradient Problem

- Ex-
- Suppose an RNN is trying to learn dependencies between the first and the 100th word of a sentence.
- As the gradient is propagated back from the 100th to the 1st word, the gradients diminish, making it difficult for the RNN to learn long-term dependencies.
- The network can only focus on short-term patterns, and the gradient effectively becomes 0, preventing meaningful updates to the earlier layers.

Long Short-Term Memory (LSTM)

- The Long Short-Term Memory (LSTM) is the advanced type of RNN, which was designed to prevent both decaying and exploding gradient problems.
- Just like RNN, LSTM has repeating modules, but the structure is different. Instead of having a single layer of tanh, LSTM has four interacting layers that communicate with each other.
- This four-layered structure helps LSTM retain long-term memory and can be used in several sequential problems including machine translation, speech synthesis, speech recognition, and handwriting recognition.

Long Short-Term Memory (LSTM)



More Versions in RNN

- Gated Recurrent Unit (GRU)
- Bidirectional RNN
- Attention Mechanism
- Transformer Networks